
Staff Engineer Depth

Patterns, Internals & Mental Models

Apple Staff / Principal Engineer Interview Preparation

System Design Series — Printer-Friendly Edition

Staff Engineer Depth — Patterns, Internals & Mental Models

*The vocabulary, patterns, and deep technical concepts that separate Staff from Senior engineers.
Each section: what it is → why it matters → trade-offs → when to use it → interview application.*

Table of Contents

Distributed Systems Patterns

1. [Scatter + Gather](#1-scatter--gather)
2. [Saga Pattern (Distributed Transactions)](#2-saga-pattern-distributed-transactions)
3. [Outbox Pattern](#3-outbox-pattern)
4. [Event Sourcing](#4-event-sourcing)
5. [CQRS (Command Query Responsibility Segregation)](#5-cqrs)
6. [Sidecar Pattern](#6-sidecar-pattern)
7. [Circuit Breaker](#7-circuit-breaker)
8. [Bulkhead Pattern](#8-bulkhead-pattern)
9. [Two-Phase Commit (2PC) vs Saga](#9-two-phase-commit-2pc-vs-saga)
10. [Consistent Hashing](#10-consistent-hashing)

Storage Internals

11. [LSM Tree Internals](#11-lsm-tree-internals)
12. [Database Storage: Why Fewer Large Files Beat Many Small Files](#12-database-storage-why-fewer-large-files-beat-many-small-files)
13. [B-Tree vs LSM Tree Deep Comparison](#13-b-tree-vs-lsm-tree-deep-comparison)
14. [Write-Ahead Log (WAL)](#14-write-ahead-log-wal)
15. [Bloom Filters](#15-bloom-filters)
16. [Columnar Storage (Parquet / ORC)](#16-columnar-storage-parquet--orc)

Messaging & Streaming

17. [Kafka Internals & Key Differentiators](#17-kafka-internals--key-differentiators)
18. [Kafka vs RabbitMQ vs SQS](#18-kafka-vs-rabbitmq-vs-sqs)
19. [Exactly-Once Delivery — The Hard Problem](#19-exactly-once-delivery--the-hard-problem)
20. [Backpressure](#20-backpressure)

Networking & Protocol Internals

21. [Zero-Copy I/O](#21-zero-copy-io)
22. [HTTP/2 vs HTTP/3 (QUIC)](#22-http2-vs-http3-quic)
23. [gRPC Internals](#23-grpc-internals)
24. [TCP vs UDP — When Engineers Get This Wrong](#24-tcp-vs-udp--when-engineers-get-this-wrong)
25. [Connection Pooling](#25-connection-pooling)

Consistency & Concurrency

26. [CAP Theorem — The Nuanced View](#26-cap-theorem--the-nuanced-view)
27. [PACELC — The Better Model](#27-pacelc--the-better-model)
28. [Vector Clocks & Causal Consistency](#28-vector-clocks--causal-consistency)
29. [CRDTs](#29-crdts)
30. [Optimistic vs Pessimistic Locking](#30-optimistic-vs-pessimistic-locking)

Performance & Systems

31. [CPU Cache Hierarchy & Cache-Friendly Code](#31-cpu-cache-hierarchy--cache-friendly-code)
32. [Memory-Mapped Files (mmap)](#32-memory-mapped-files-mmap)
33. [Copy-on-Write (COW)](#33-copy-on-write-cow)
34. [Epoll / kqueue — I/O Multiplexing](#34-epoll--kqueue--io-multiplexing)
35. [JVM GC — What a Senior Engineer Must Know](#35-jvm-gc--what-a-senior-engineer-must-know)

1. Scatter + Gather

What It Is

A parallel fan-out + aggregation pattern. A coordinator sends requests to N workers simultaneously (scatter), waits for responses, then merges results (gather).

```
Request ■■■ → Worker 1 (search shard A) ■■  
Request ■■■ → Scatter    ■■ → Worker 2 (search shard B) ■■■■ → Gather    ■■ → Response  
Request ■■■ → Worker 3 (search shard C) ■■
```

When to Use

- Search across sharded indexes (each shard holds a subset of data)
- Distributed aggregation (compute sum/avg across many nodes)
- Fan-out reads where you need data from multiple services

Real Example: Apple Maps Search

User searches "coffee near me":

1. Scatter: query sent to all 20 geographic shards in parallel
2. Each shard returns top-10 local results with a score
3. Gather: merge-sort all responses by score, return top-10 global results

Trade-offs

Concern	Detail
Latency	Response time = slowest worker (tail latency problem)
Partial failure	What if 1 of 20 shards is down? Return partial results? Error?
Amplification	1 request → N backend calls. Fan-out ratio must be bounded.
Hedging	Send duplicate requests to 2 workers for same shard; take first response (reduces tail latency at cost of double load)

Tail Latency Mitigation — Hedged Requests

At t=0ms: send request to worker
 At t=95th-percentile (e.g., 50ms): if no response yet, send duplicate request to another replica
 Return whichever responds first
 Cancel the other

Cost: ~5% extra load
 Benefit: P99 latency drops to ~P50 of original distribution

This is used by Google Bigtable, Cassandra, and Spanner.

Advantages vs Disadvantages

Advantages	Disadvantages
Parallelism: N workers run simultaneously → response time = 1 worker, not N	Tail latency: overall latency = slowest worker; one slow shard poisons the whole request
Horizontal scalability: add shards to increase throughput without changing coordinator	Fan-out amplification: 1 client request becomes N backend calls; at 1000 req/s × 20 shards = 20,000 backend calls/s
Partial results possible: can return top-K even if some shards are down	Partial failure handling complexity: do you error or return incomplete data when 1 of 20 shards fails?

Advantages	Disadvantages
Natural fit for sharded data: each shard only processes its own data slice	Merge overhead: coordinator must sort/aggregate N result sets; can become a CPU bottleneck at high fan-out
Fault isolation: one shard crashing doesn't block others	Increased network usage: N outbound + N inbound connections per request
Load distribution: sharding distributes work evenly across fleet	Result consistency: if shards have different data freshness, merged result may be inconsistent

Interview Application

Mention Scatter+Gather when designing: distributed search, recommendation systems, map reduce, any sharded read-aggregation pattern.

2. Saga Pattern (Distributed Transactions)

The Problem

A business transaction spans multiple microservices. You need atomicity (all or nothing) but can't use a database transaction across services.

Example: Place order = [Reserve inventory] + [Charge payment] + [Schedule delivery]

All three must succeed or all must be rolled back.

Choreography-Based Saga

```
OrderService → publishes "OrderCreated" event
InventoryService → listens → reserves stock → publishes "StockReserved"
PaymentService → listens → charges card → publishes "PaymentCharged"
DeliveryService → listens → schedules delivery
```

```
On failure at any step:
PaymentService fails → publishes "PaymentFailed"
InventoryService → listens → releases stock (compensating transaction)
OrderService → listens → marks order as failed
```

Pros: Decoupled, services don't know about each other

Cons: Hard to reason about overall state, hard to debug cascading failures

Orchestration-Based Saga

```

SagaOrchestrator → calls InventoryService → on success
                  → calls PaymentService   → on success
                  → calls DeliveryService   → on success → done

On failure:
SagaOrchestrator → calls PaymentService.compensate()
                  → calls InventoryService.compensate()

```

Pros: Centralized state, easy to monitor and debug

Cons: Orchestrator becomes a coordination bottleneck; single point of failure

Key Insight: Compensating Transactions, Not Rollbacks

Sagas don't roll back — they **compensate**. "Cancel reservation" is a new transaction that undoes the previous one. This means:

- Compensation must be idempotent
- There's a window where partial state is visible (eventual consistency)
- Some compensations can't be undone (e.g., "email sent" — you can send a follow-up but not un-send)

Saga: Overall Advantages vs Disadvantages

<i>Advantages</i>	<i>Disadvantages</i>
Works across microservices with separate databases	No true atomicity — partial state is visible to other transactions during execution
High availability: no distributed locks held; services can fail independently	No isolation: another process can read "half-complete" order state while Saga is in progress
Lower latency than 2PC: no lock-hold round trips	Compensating transactions can fail too — requires retry and fallback logic
Each step is a local ACID transaction — reliable at the individual service level	"Cannot compensate" cases (email sent, SMS sent) require explicit handling (e.g., send correction)
Services are loosely coupled (choreography) or clearly orchestrated	Debugging is harder — distributed trace needed to follow a saga across 5 services
Scales well — no coordinator bottleneck (choreography)	Eventual consistency window is application-visible; users may see inconsistent state briefly

3. Outbox Pattern

The Problem

You need to atomically:

1. Save data to your database
2. Publish an event to Kafka/event bus

What if the DB write succeeds but Kafka publish fails? Or vice versa?

Solution: Transactional Outbox

```
Within the same DB transaction:
  INSERT INTO orders (id, status) VALUES (...)
  INSERT INTO outbox (id, event_type, payload, status) VALUES (...)
COMMIT
```

```
A separate "outbox poller" process:
  SELECT * FROM outbox WHERE status = 'PENDING'
For each row:
  Publish to Kafka
  UPDATE outbox SET status = 'SENT'
```

Now atomicity is guaranteed by the DB transaction. The outbox poller retries until Kafka publish succeeds.

Change Data Capture (CDC) — Better Variant

Instead of polling, use **Debezium** to read the DB's WAL (transaction log):

```
PostgreSQL WAL → Debezium → Kafka
```

Zero additional writes to DB. Events published in the same order they were written.

Why This Matters at Scale

Without outbox pattern, you get **dual-write inconsistency**:

- "Write to DB succeeded, Kafka failed" → DB and downstream services diverge
- "Write to Kafka succeeded, DB failed" → phantom events for something that never happened

This is a **common senior interview miss** — many candidates say "write to DB and Kafka in the same request" without recognizing the atomicity problem.

Advantages vs Disadvantages

<i>Advantages</i>	<i>Disadvantages</i>
Guarantees atomicity between DB write and event publish — no dual-write inconsistency	Extra table in DB (outbox) adds write overhead per transaction

<i>Advantages</i>	<i>Disadvantages</i>
At-least-once delivery: poller retries until Kafka ACKs — no message loss	Polling adds latency: poller runs every N seconds → up to N seconds before event is published
Works with any database that supports transactions (PostgreSQL, MySQL)	Outbox table grows unless purged; requires a separate cleanup job
CDC variant (Debezium) eliminates polling latency and extra DB writes	CDC adds operational complexity: Debezium, WAL access, connector management
Idempotent consumers handle the occasional duplicate from at-least-once delivery	Requires consumers to be idempotent — adds consumer-side complexity
Simple mental model: event is visible to downstream only after DB commit succeeds	Outbox poller is another process to deploy, monitor, and operate

4. Event Sourcing

What It Is

Instead of storing current state, store **the sequence of events that led to that state**.

Traditional: users table → {id: 1, balance: 150}

Event sourcing: events table

```
{user_id: 1, type: "AccountOpened", amount: 200, ts: t1}
{user_id: 1, type: "MoneyDebited", amount: 50, ts: t2}
{user_id: 1, type: "MoneyDebited", amount: 0, ts: t3}
Current balance = replay all events = 150
```

Benefits

- Complete audit log^{**}: know not just current state but how you got there
- Temporal queries^{**}: "What was the balance at time T?"
- Event replay^{**}: rebuild read models by replaying events
- Decoupling^{**}: multiple projections from same event stream (balance, transaction history, fraud alerts)

Challenges

- Read performance: **replaying all events for every read is slow** → use snapshots^{**} (checkpoint current state periodically, replay from snapshot)
- Schema evolution^{**}: old events must remain valid as schema changes
- Eventual consistency^{**}: read model is a projection, may lag behind events

Snapshots

```
Every 1000 events, store snapshot:  
{user_id: 1, balance: 150, snapshot_at_sequence: 1000}  
  
On read: load snapshot, replay only events after sequence 1000
```

When to Use (and Not Use)

Use when: audit trail is critical (financial, healthcare), multiple views of same data, temporal queries

Don't use when: simple CRUD, team lacks EventSourcing experience, simple state sufficient

Advantages vs Disadvantages

<i>Advantages</i>	<i>Disadvantages</i>
Complete, immutable audit trail — know exactly how state was reached	Read performance: replaying N events per query is slow without snapshots
Temporal queries: "what was the state at time T?" are trivially answered	Storage grows unboundedly — every change is stored forever (mitigated by snapshots + compaction)
Multiple independent projections from one event stream (balance, fraud alerts, analytics)	Schema evolution is hard: old events must remain parseable as schema changes
Bug recovery: replay events with a fixed handler to recompute correct state	High learning curve — most engineers are not familiar with event sourcing patterns
Natural fit for event-driven architectures and CQRS	Eventual consistency: read projections lag behind the event log
Easy undo: "delete" is just appending a compensating event, not mutating data	Querying current state requires either a projection or full replay — no simple SELECT
Decouples write model from read model naturally	Snapshots add complexity: when to snapshot, how to migrate snapshots on schema change

5. CQRS

What It Is

Command Query Responsibility Segregation — separate the write model (commands) from the read model (queries).

```
Write side (Command):
  POST /orders → OrderService → writes to normalized DB (PostgreSQL)
                                → emits OrderCreated event

Read side (Query):
  GET /orders/summary → OrderReadService → reads from denormalized view (Elasticsearch, Redis, read replica)
                                          → view kept up to date by consuming events
```

Why It Matters

- Read and write patterns are fundamentally different
 - Writes: transactional, normalized, consistency critical
 - Reads: high volume, need denormalized/aggregated data, can tolerate slight staleness
- CQRS lets you **scale read and write independently**
- Read model can be rebuilt from scratch by replaying events (pairs well with Event Sourcing)

Advantages vs Disadvantages

<i>Advantages</i>	<i>Disadvantages</i>
Read and write sides scale independently — add read replicas without touching write path	Two models to maintain: schema changes require updating both command and query sides
Read model can be optimized for query patterns (denormalized, pre-aggregated)	Eventual consistency between write and read model — reads may be milliseconds to seconds stale
Write model stays normalized and consistent — no query optimization compromises	Increased infrastructure: separate read store (Redis, Elasticsearch) plus sync mechanism
Rebuild read model from scratch by replaying events — no data loss on view changes	More complex deployment and operational surface area
Supports multiple different read models from the same write store	Overkill for simple CRUD applications — adds complexity without proportional benefit
Dramatically reduces read latency — reads hit cache/denormalized view, not normalized DB	Developer mental overhead: engineers must understand which model to write/read from

6. Sidecar Pattern

What It Is

Deploy a helper container alongside your main application container (in the same pod/VM). The sidecar handles cross-cutting concerns.

```
[App Container]  [Sidecar Container]
  ■              ■
Business Logic  Service Mesh Proxy (Envoy)
               - TLS termination
               - mTLS between services
               - Traffic routing
               - Metrics collection
               - Circuit breaking
```

Examples

- Envoy sidecar (Istio)**: intercepts all network traffic to/from app, handles load balancing, retries, circuit breaking
- Log shipper sidecar**: app writes logs to stdout; sidecar tails and ships to Splunk/CloudWatch
- Secret injector sidecar**: fetches secrets from Vault; writes to shared memory volume

Why Staff Engineers Know This

At scale, you can't add service mesh features to every application. The sidecar pattern injects cross-cutting concerns without modifying application code. This enables:

- Zero-trust networking (mTLS everywhere without app changes)
- Consistent observability
- Traffic management (canary deploys, A/B routing)

Advantages vs Disadvantages

<i>Advantages</i>	<i>Disadvantages</i>
No application code changes: inject TLS, retries, circuit breaking, metrics transparently	CPU + memory overhead: Envoy sidecar adds ~50–150MB RAM and 5–10% CPU per pod
Consistent cross-cutting behavior across all services regardless of language/framework	Latency overhead: every request proxied through sidecar adds ~0.5–2ms
Centralized policy: update mTLS policy once, propagated to all sidecars automatically	Operational complexity: Istio control plane (istiod) is complex to operate and debug
Enables zero-trust networking without rewriting applications	Sidecar startup ordering: app container must wait for sidecar to be ready — complicates init
Traffic management (canary, A/B, traffic splitting) without code changes	Debugging is harder: network issues appear in sidecar logs, not app logs

<i>Advantages</i>	<i>Disadvantages</i>
Upgradable independently: update sidecar version without redeploying app	Not all workloads benefit: high-frequency internal RPCs may not need full service mesh overhead

7. Circuit Breaker

The Problem

Service A calls Service B. Service B is slow/erroring. Without a circuit breaker, A's threads pile up waiting for B, consuming resources until A itself falls over. This is **cascading failure**.

States

```
CLOSED (normal):
- Requests pass through to Service B
- Track failure rate over rolling window

OPEN (failure threshold exceeded):
- Requests fail fast (return error immediately without calling B)
- Timer starts

HALF-OPEN (probing):
- Allow a few requests through to test if B recovered
- If success → CLOSED
- If fail → OPEN again
```

Implementation Detail

```
// Simplified circuit breaker logic
class CircuitBreaker {
    int failureCount;
    int threshold = 5;
    long lastFailureTime;
    long timeout = 60_000; // 60s open window
    State state = CLOSED;

    Result call(Supplier<Result> fn) {
        if (state == OPEN) {
            if (now() - lastFailureTime > timeout) state = HALF_OPEN;
            else throw new CircuitOpenException();
        }
        try {
            Result r = fn.get();
            if (state == HALF_OPEN) state = CLOSED; // recovered
            failureCount = 0;
            return r;
        } catch (Exception e) {
            failureCount++;
            lastFailureTime = now();
            if (failureCount >= threshold) state = OPEN;
            throw e;
        }
    }
}
```

Hystrix → Resilience4j

Netflix Hystrix (deprecated) was the canonical circuit breaker library. Resilience4j is the modern replacement with reactive support.

Advantages vs Disadvantages

Advantages	Disadvantages
Prevents cascading failures: a slow downstream service doesn't kill the caller	False positives: circuit opens during a brief spike, rejecting requests that would have succeeded
Fail fast: instead of waiting 30s for timeout, returns error in < 1ms when circuit is open	Threshold tuning is tricky: too sensitive = frequent false opens; too lenient = slow to protect
Self-healing: HALF-OPEN state probes for recovery automatically	Adds per-call overhead: failure counting and state checking on every request
Improves resource utilization: no threads wasted waiting on a known-bad dependency	Requires fallback logic: what do you return when the circuit is open? Must be designed upfront
Provides visibility: circuit state is a clear signal for monitoring and alerting	Distributed systems: each service instance has its own circuit — no shared state (unless using Redis)

Advantages	Disadvantages
Reduces load on failing service: stops hammering it while it's recovering	Non-obvious behavior for clients: "circuit open" errors look like random failures without good error messaging

8. Bulkhead Pattern

What It Is

Isolate resources for different consumers so one doesn't starve the others. Named after ship compartments that prevent flooding from sinking the whole vessel.

```
Without bulkheads:
[Thread Pool: 100 threads]
  ← All requests (search, checkout, homepage) share the pool
  ← Slow search query consumes 100 threads → checkout is blocked

With bulkheads:
[Thread Pool A: 40 threads] → Search API calls
[Thread Pool B: 40 threads] → Checkout API calls
[Thread Pool C: 20 threads] → Homepage API calls
  ← Slow search consumes 40 threads, checkout pool unaffected
```

Application

- Separate connection pools per downstream service
- Separate thread pools per traffic class (critical vs. best-effort)
- Separate queue capacity per tenant (multi-tenant SaaS)

Advantages vs Disadvantages

Advantages	Disadvantages
Fault isolation: one slow/failing dependency consumes only its allocated pool, not the whole system	Resource underutilization: if checkout pool is idle and search pool is saturated, idle threads can't help
Prevents noisy-neighbor problem in multi-tenant systems	Pool sizing is difficult: requires capacity modeling per workload; wrong sizes reduce effectiveness
Critical paths (checkout, payment) are protected from degradation by non-critical paths (search, recs)	More operational complexity: more thread pools, connection pools to monitor and tune

Advantages	Disadvantages
Predictable behavior under partial failure: one component degrading doesn't cascade	Memory overhead: each isolated pool has its own queue and thread stacks
Easier capacity planning: size each bulkhead independently for its specific workload	Must identify correct isolation boundaries upfront — wrong boundaries give false sense of safety
Can be used at multiple levels: threads, connections, queue depth, memory	Too many fine-grained bulkheads increases complexity without proportional safety gain

9. Two-Phase Commit (2PC) vs Saga

2PC

Phase 1 – Prepare:
 Coordinator → all participants: "Can you commit?"
 Each participant: acquires locks, writes to WAL, responds "Yes" or "No"

Phase 2 – Commit:
 If all said Yes → Coordinator: "Commit"
 Each participant: commits, releases locks
 If any said No → Coordinator: "Abort"

Problems with 2PC:

- Blocking protocol**: if coordinator crashes after Phase 1, participants hold locks indefinitely
- Single point of failure**: coordinator crash = system blocked
- Not suitable for microservices**: requires all participants to be available simultaneously

2PC vs Saga Comparison

Aspect	2PC	Saga
Atomicity	True ACID atomicity	Eventual (compensating txns)
Isolation	Full isolation during lock hold	No isolation (partial state visible)
Availability	Blocked if coordinator fails	High availability
Latency	Higher (2 round trips + locking)	Lower
Use case	Same DB/resource manager	Cross-service/microservice
Suitable for microservices	No	Yes

10. Consistent Hashing

The Problem

You have 3 cache nodes. You use $\text{key} \% 3$ to route. You add a 4th node. Now $\text{key} \% 4$ remaps **75% of keys** — massive cache miss storm.

Solution: Consistent Hash Ring

```
Hash space: [0, 2^32)
Ring: wrap around (0 connects to 2^32)
Nodes hash to positions on ring
Key maps to nearest node clockwise

Adding a node: only keys between new node and its predecessor remap (~1/N fraction)
Removing a node: only that node's keys remap to next node

Virtual nodes (vnodes): each physical node occupies multiple ring positions
→ More uniform distribution
→ Better load balancing when node capacities differ
```

Why Virtual Nodes?

Without vnodes, 3 nodes may hash to positions [10%, 30%, 90%] on the ring, causing unequal load. With 150 vnodes per physical node, distribution converges to uniform by the law of large numbers.

Advantages vs Disadvantages

<i>Advantages</i>	<i>Disadvantages</i>
Minimal key remapping on node add/remove: only $\sim 1/N$ keys move (vs $\sim N-1/N$ with modulo hashing)	More complex than simple modulo hashing — requires understanding ring and vnodes
No mass cache invalidation storms when cluster topology changes	Virtual nodes increase metadata: with 150 vnodes $\times$ N nodes, ring can have thousands of entries
Scales horizontally: add nodes incrementally without full resharding	Hotspot risk without vnodes: uneven ring positions $\rightarrow$ uneven load distribution
Widely used in production systems: Cassandra, DynamoDB, Redis Cluster all use variants	Node heterogeneity handling: different-capacity nodes need different vnode counts — complex to manage

Advantages	Disadvantages
Works well for both caching and data sharding use cases	Consistent hashing does not guarantee perfect uniformity — statistical distribution, not deterministic
Ring lookup is $O(\log N)$ with sorted vnode list + binary search	Rebalancing still requires data transfer when nodes join/leave (just less of it)

11. LSM Tree Internals

Write Path (Why LSM is Fast for Writes)

1. Write to WAL (sequential disk write – very fast, $O(1)$)
2. Write to MemTable (in-memory sorted structure, typically a red-black tree or skip list)
3. When MemTable hits size threshold (e.g., 64MB):
 - Flush to disk as an SSTable (Sorted String Table)
 - SSTable is immutable, sorted by key, with an index and Bloom filter
4. Background compaction merges SSTables, removes tombstones and old versions

Key insight: All writes are sequential (WAL + SSTable flush). Sequential I/O is **10–100x faster** than random I/O on spinning disks, and even on SSDs sequential is faster due to write amplification.

Read Path

1. Check MemTable ($O(\log n)$, most recent writes)
2. Check each SSTable level from newest to oldest:
 - a. Check Bloom filter – skip SSTables where key definitely absent
 - b. Check SSTable index – binary search for key position
 - c. Read block from disk

Read amplification: In worst case, check L levels of SSTables. Bloom filters reduce this dramatically.

Compaction Strategies

Size-Tiered Compaction (Cassandra default)

When N SSTables of similar size accumulate → merge into 1 larger SSTable
 Simple, good for write-heavy workloads
 Drawback: space amplification (old + new SSTables exist during merge)

Leveled Compaction (RocksDB default)

L0: unsorted SSTables (from MemTable flushes)
 L1: sorted, limited total size (e.g., 100MB)
 L2: 10x L1 size (1GB)
 L3: 10x L2 (10GB)
 ...

Key range overlap only within same level (except L0)
 Reads hit at most 1 SSTable per level (except L0)
 Better read performance, less space amplification
 More I/O during compaction (write amplification ~10–30x)

Bloom Filter Integration

Each SSTable has a Bloom filter: "Does key X exist in this SSTable?"
 Bloom filter: space-efficient probabilistic structure

- False negatives: impossible (if key is in SSTable, filter says yes)
- False positives: possible but rare (~1% with good sizing)

Effect: 99% of "key not found" queries avoid disk reads entirely

LSM Tree: Advantages vs Disadvantages

<i>Advantages</i>	<i>Disadvantages</i>
Extremely fast sequential writes: all writes are sequential (WAL + SSTable append)	Write amplification: leveled compaction rewrites data 10–30x; size-tiered 5–10x
High write throughput: can absorb millions of writes/sec on commodity hardware	Read amplification: a key lookup may check MemTable + multiple SSTable levels
Good compression: SSTables stored in sorted, compressed blocks (10–20x ratio)	Compaction I/O competes with live read/write traffic — can cause latency spikes
Works well with sequential storage (HDDs and SSDs)	Space amplification during compaction: old + new SSTables coexist temporarily (up to 2x space)
Bloom filters make "key not found" reads nearly free (no disk I/O)	MemTable crash without WAL = data loss; WAL adds write latency on fsync
Immutable SSTables: no in-place updates → no corruption risk from partial writes	Compaction is a background tax: must be tuned carefully to avoid impacting foreground latency
Efficient range scans on recently written data (MemTable is sorted)	Not ideal for read-heavy, low-write workloads — B-Tree has lower read amplification

12. Database Storage: Why Fewer Large Files Beat Many Small Files

The Core Problem with Many Small Files

Filesystem metadata overhead:

Every file on disk requires an **inode** (index node) — a data structure storing metadata (permissions, timestamps, size, pointers to data blocks). On ext4, each inode is 256 bytes. With millions of small files:

```
10M small files × 256 bytes inode = 2.56 GB just for inodes
Inode table loaded into memory → competes with actual data cache
Directory traversal: O(n) per directory with many entries
```

Block allocation fragmentation:

Disk storage is managed in **blocks** (typically 4KB). A 1KB file still occupies one full 4KB block — **75% wasted**. With millions of 1KB files: massive internal fragmentation.

Read amplification:

To read a 10MB logical file stored as 10,000 × 1KB files:

- 10,000 metadata lookups (inode reads)
- 10,000 data block reads, potentially non-contiguous
- Many disk seeks (especially on HDDs) vs. one sequential read for a single 10MB file

Page cache inefficiency:

OS page cache (buffer cache) is managed at the block level. 10,000 files = 10,000 separate cache entries, each evictable independently → more cache pressure.

Why Database Engines Use Large Files

Databases like RocksDB, Cassandra, PostgreSQL, and Kafka all write large segment/SSTable/WAL files:

```
Kafka: Partition logs are large, append-only segment files (default 1GB)
RocksDB SSTables: Each file is hundreds of MB to GB
PostgreSQL: 1GB relation files (default segment size)
```

Reasons:

1. **Sequential I/O**: One large file can be read sequentially, saturating disk bandwidth. Random small file reads = seeks + rotational latency.
2. **Prefetching**: OS read-ahead works well with large sequential files; ineffective with many small files.
3. **Compaction/merge efficiency**: Merging 2 large SSTables = 2 sequential scans + 1 sequential write. Merging 1000 small files = 1000 seeks.

4. **mmap efficiency:** Memory-mapping a large file creates one VM region. Mapping 10,000 small files = 10,000 VM regions, TLB pressure.
5. **fsync cost:** Each `fsync()` syscall flushes one file's dirty pages. Fewer files = fewer fsyncs needed for durability.

The Trade-offs

<i>Dimension</i>	<i>Many Small Files</i>	<i>Fewer Large Files</i>
Sequential read bandwidth	Poor (seeks dominate)	Excellent
Random access to a specific record	Good (open specific file)	Requires internal index
Metadata overhead	High (inode per file)	Low
Deletion/GC	Easy (delete file)	Need tombstones or GC pass
Recovery time	Fast (restart clean)	May need to scan large file
Write performance	Poor (many fsyncs)	Good (one fsync per large file)
Caching efficiency	Poor (many cache entries)	Good (large contiguous regions)
Partial failure	Isolated to one file	One bad file → more data at risk

When Small Files Are Acceptable

- Each file is independently addressable and accessed randomly (e.g., one file per user profile)
- Write-once, read-many (immutable content-addressed blobs in object storage)
- NFS/distributed FS where metadata is centralized (HDFS minimizes this)

HDFS Design Philosophy

HDFS explicitly addresses this with a **large block size (128MB default, vs ext4's 4KB)**:

Small file problem in HDFS: each file gets its own block allocation
 → 1KB file stored in HDFS uses 128MB of "logical" space
 → NameNode memory proportional to number of files, not total size

Solution: SequenceFile, Avro, Parquet – pack many logical records into few large files

13. B-Tree vs LSM Tree Deep Comparison

B-Tree

On-disk balanced tree. All operations $O(\log N)$.
Reads/writes both modify tree in place.
Pages: typically 4KB-16KB, aligned to disk blocks.

Write: find leaf page, modify in place, write page back
→ Random write (seek to page location)
→ WAL write first for crash recovery (extra I/O)
→ Page splits when leaf overflows (cascading updates up tree)

Amplification factors:

- Write amplification: typically 2–5x (WAL + page write, page splits)
- Read amplification: $O(\log N)$ page reads for a lookup
- Space amplification: ~30–50% free space in pages for future insertions (B-Tree fragmentation)

LSM Tree

Writes are sequential; reads may require multiple levels.

Write amplification: higher (compaction rewrites data multiple times)
Leveled compaction: ~10–30x write amplification
Size-tiered: ~5–10x

Read amplification: higher than B-Tree in worst case
Bloom filters reduce this dramatically

Space amplification:
Temporary: during compaction, old + new SSTables coexist
Post-compaction: better than B-Tree (no fragmentation within SSTables)

Decision Guide

<i>Workload</i>	<i>Choose</i>
Write-heavy, streaming ingestion	LSM (Cassandra, RocksDB)
Read-heavy, OLTP	B-Tree (PostgreSQL, MySQL)
Time-series data	LSM (InfluxDB, TimescaleDB)
General purpose SQL	B-Tree
Key-value with high write throughput	LSM (RocksDB, LevelDB)
Need range scans on sorted keys	Both support, LSM slightly better for recent data

B-Tree vs LSM: Head-to-Head Advantages/Disadvantages

<i>Dimension</i>	<i>B-Tree Advantage</i>	<i>LSM Advantage</i>
Read latency	✓ Lower read amplification ($O(\log N)$ pages)	— Higher in worst case
Write throughput	— Random writes, limited by seek latency	✓ Sequential writes, 10–100x faster
Write amplification	✓ Lower (2–5x)	— Higher (10–30x leveled)
Space amplification	— 30–50% page fragmentation	✓ Better post-compaction (no fragmentation)
Point lookup	✓ Efficient (tree traversal)	— Must check multiple SSTables
Range scan	✓ Efficient (in-order tree traversal)	✓ Efficient for recent data (MemTable + L0)
Crash recovery	✓ WAL + page consistency	✓ WAL + immutable SSTables
Compaction impact	✓ None (in-place updates)	— Compaction I/O spikes
Compression	— Row-oriented pages, lower compression	✓ Block-compressed SSTables
Concurrent writes	— Page-level locking limits concurrency	✓ MemTable absorbs bursts

14. Write-Ahead Log (WAL)

What It Is

Before any data page modification, write the change to a sequential log file first. Only after the log write is durable (fsync) can the in-memory change be made and acknowledged.

Why It Works

Without WAL:

DB crashes mid-write → partial write to data file → corruption

With WAL:

1. Write operation to WAL (sequential, fast)
2. fsync WAL → durable
3. Acknowledge write to client
4. Later: apply WAL record to data file (can be async)

After crash: replay WAL from last checkpoint → fully recover

Partial WAL writes: detected via checksum in WAL record

WAL in Different Systems

System	WAL Name	Notes
PostgreSQL	pgwal (formerly pgxlog)	Used for replication too (streaming replication)
MySQL InnoDB	Redo log	Fixed-size circular log
Cassandra	Commit log	Per-node, flushed with MemTable
Kafka	Actually IS a log	The "log" is the primary storage, not just a WAL
ZooKeeper	Transaction log	Drives leader election consistency

WAL for Replication

PostgreSQL streaming replication works by shipping WAL records to replicas:

Primary: writes WAL → replica reads WAL stream → applies to replica's data files
Replica is always slightly behind (replica lag = WAL shipping delay)

Advantages vs Disadvantages

Advantages	Disadvantages
Crash safety: crash after WAL write but before page write → replay WAL, no corruption	Write amplification: every write hits WAL sequentially + data file randomly = 2 I/O operations minimum
Sequential WAL writes are fast (append-only, no seeks)	WAL fsync adds latency: each commit requires fsync to guarantee durability

Advantages	Disadvantages
Enables replication: ship WAL records to replicas for exact state reproduction	WAL files consume disk space: must be retained until replicas consume them
Enables point-in-time recovery (PITR): replay WAL from base backup to any timestamp	WAL replay during recovery can be slow for large volumes of accumulated WAL
Atomic commits: transaction either fully replayed or not at all	WAL file management complexity: need to monitor WAL growth, archiving, retention policies
Decouples write acknowledgment from slow data file I/O (WAL fsync is sufficient for durability)	Synchronous replication (waiting for replica WAL ack) adds latency to every commit

15. Bloom Filters

What It Is

A space-efficient probabilistic data structure that answers: "Is element X in the set?"

- Definitive NO**: element is definitely not in the set
- Probable YES**: element might be in the set (false positive possible)

False negatives: **impossible**

False positives: tunable, typically 1%

How It Works

```
k hash functions, bit array of size m

Insert "apple":
  hash1("apple") = 3  → set bit[3] = 1
  hash2("apple") = 7  → set bit[7] = 1
  hash3("apple") = 12 → set bit[12] = 1

Query "apple":
  Check bits 3, 7, 12 → all 1 → "Probably YES"

Query "orange":
  hash1("orange") = 3  → bit[3] = 1
  hash2("orange") = 15 → bit[15] = 0 → "Definitely NO"
```

Space Efficiency

For n elements with false positive rate p:

Optimal bit array size: $m = -n \times \ln(p) / (\ln(2))^2$
Optimal hash functions: $k = (m/n) \times \ln(2)$

Example: 1M elements, 1% FPR

$m = 9.6\text{M bits} = 1.2\text{ MB}$ (vs 8MB for a hash set of the same keys)

Applications at Scale

- RocksDB/Cassandra**: One Bloom filter per SSTable → 99% of "key not found" disk reads eliminated
- Chrome Safe Browsing**: List of malicious URLs stored as Bloom filter on device
- Akamai CDN**: Avoid caching one-hit-wonders (URLs requested only once filtered with Bloom filter)
- Databases**: Join optimization (build Bloom filter on small table, probe for each row of large table)

Advantages vs Disadvantages

<i>Advantages</i>	<i>Disadvantages</i>
Extremely space-efficient: 1.2 MB for 1M elements at 1% FPR vs 8 MB for a hash set	False positives: 1% of "not in set" answers will be wrong — must design for this
$O(k)$ lookup time (k = number of hash functions, typically 3–10) — constant time	False negatives impossible, but false positive rate grows as the filter fills
No deletion support: cannot remove elements from a standard Bloom filter	Counting Bloom filters support deletion but require more space (4 bits per bucket vs 1 bit)
Dramatically reduces disk I/O for "key not found" in databases (99% reduction)	FPR degrades if more elements added than designed for — must size correctly upfront
Can fit entirely in CPU cache for small sets → extremely fast in practice	Not suitable when false positives are unacceptable (e.g., "is this user banned?" — must use exact set)
Simple to implement; language-agnostic; no external dependencies	Requires knowing approximate set size in advance to size correctly

16. Columnar Storage (Parquet / ORC)

Row vs Column Storage

```

Row storage (traditional DB):
[user_id=1, name="Alice", age=30, email="alice@x.com"]
[user_id=2, name="Bob", age=25, email="bob@x.com"]

Column storage:
user_id column: [1, 2, 3, ...]
name column: ["Alice", "Bob", ...]
age column: [30, 25, ...]
email column: ["alice@x.com", "bob@x.com", ...]

```

Why Columnar Is Better for Analytics

Query: "What is the average age of all users?"

```

Row storage: read ALL columns for ALL rows → then extract age
Column storage: read ONLY the age column → directly compute average

If a row is 100 bytes and age is 4 bytes:
Row storage: read 100 bytes × N rows
Column storage: read 4 bytes × N rows = 25x less I/O

```

Compression:

Columns have homogeneous data → higher compression:

- Integer age column: dictionary encode (30 → 0, 25 → 1) → delta encode → bit-pack
- String column: dictionary encoding (same string stored once)
- Run-length encoding: [30, 30, 30, 31] → [(30, 3), (31, 1)]

Vectorized execution:

Modern CPUs have SIMD instructions (AVX2, AVX-512) that operate on 256–512 bit registers.

- 8 int32 values processed in a single AVX2 ADD instruction
- Columnar format aligns naturally with this (array of values, not interleaved rows)

When to Use Row vs Columnar

<i>Pattern</i>	<i>Storage</i>
OLTP (high-frequency inserts, point reads)	Row (PostgreSQL, MySQL)
OLAP (analytics, aggregations, scans)	Columnar (Parquet, BigQuery, Redshift)
Time-series analytics	Columnar (InfluxDB IOx uses Apache Arrow)
Machine learning training data	Columnar (Parquet is standard)

Columnar Storage: Advantages vs Disadvantages

Advantages	Disadvantages
Massive I/O reduction for analytical queries: only read columns needed (e.g., 4 bytes vs 100 bytes per row)	Poor for OLTP: inserting a single row requires updating every column file/stripe
Superior compression: homogeneous column data compresses 10–20x better than row-mixed data	Poor for point lookups: retrieving one full record requires reading from N column files
Vectorized SIMD execution: CPU processes 8–16 column values per instruction	Row reconstruction expensive: joining N columns back into a row for each result
Predicate pushdown: skip entire row groups based on min/max statistics (no I/O)	Append-heavy workloads: columnar formats like Parquet are immutable — updates require rewriting files
Partition pruning: queries skip entire files/partitions based on partition columns	Higher write latency: must buffer enough data to form column groups before writing efficiently
Schema evolution supported: add new columns without rewriting existing data (in Parquet)	Not suitable for streaming row-level updates — designed for batch analytics workloads

17. Kafka Internals & Key Differentiators

Kafka's Core Architecture

```

Topic: "orders"
  Partition 0: [msg_0, msg_1, msg_2, ...] → stored on Broker 1
  Partition 1: [msg_0, msg_1, ...]       → stored on Broker 2
  Partition 2: [msg_0, msg_1, ...]       → stored on Broker 3

Producer: writes to partition (by key hash or round-robin)
Consumer Group: each partition consumed by exactly one consumer in the group
  → Parallelism = number of partitions

```

Zero-Copy I/O — Kafka's #1 Differentiator

Traditional file-to-network data path (4 copies):

1. Read from disk → OS kernel page cache (DMA copy)
 2. Copy from kernel buffer → user-space application buffer
 3. Copy from user-space → kernel socket buffer
 4. Copy from socket buffer → NIC (DMA copy)
- = 4 copies, 2 context switches

Zero-copy with `sendfile()` syscall (2 copies):

```
1. Read from disk → OS kernel page cache (DMA copy)
2. Copy directly from kernel page cache → NIC buffer (DMA copy)
= 2 copies, 0 user-space copies, 0 context switches
Application never touches the data bytes
```

Kafka uses `sendfile()` (Linux) / `TransferTo()` (Java NIO) for consumer reads:

- Result**: Kafka can saturate network bandwidth (multi-GB/s) on a single broker
- Traditional message brokers that touch data in user-space cannot compete

Sequential I/O

Kafka only does **sequential writes and reads**:

```
Producer writes: always append to end of partition log file
Consumer reads: read sequentially from offset

Sequential disk I/O on HDD: ~100-200 MB/s
Random I/O on HDD: ~0.1-1 MB/s (seek time dominates)
Sequential I/O ≈ random I/O on SSD, but sequential is still better for write throughput
```

This is why Kafka **outperforms in-memory message queues at scale** — it saturates disk bandwidth with sequential I/O rather than bottlenecking on random access.

Batching

Kafka accumulates messages in a **record batch** before sending:

```
producer.send(msg1) ↓
producer.send(msg2) ↓ → batch.linger_ms (wait up to 5ms) → send one large batch
producer.send(msg3) ↓
```

Benefits:

- Network overhead amortized over batch
- Compression applies to whole batch (better ratio than per-message)
- One `fsync` per batch instead of per message

Page Cache as Kafka's Buffer

Kafka relies on the **OS page cache** rather than JVM heap:

```
Written messages: immediately in OS page cache (in-memory)
New consumers: often served directly from page cache (no disk read)
Consumer falling behind: served from disk, but sequential read = efficient
```

```
Kafka process restart: page cache survives (unlike JVM heap)
→ Kafka recovers quickly from restarts
```

Counter-intuitive result: Kafka with a large page cache can sustain in-memory throughput even for "disk-backed" data.

Log Compaction

Normal Kafka: retain messages for N days, then delete
Log compaction: retain the LATEST message per key, indefinitely

Useful for:

- Change data capture (latest state of a database row)
- Configuration updates (latest config version per key)
- User event stream (latest session state)

Compacted topic is a persistent, queryable key-value store backed by Kafka

Consumer Group Semantics

Topic: 4 partitions, Consumer Group: 3 consumers

Consumer A → Partition 0, 1
Consumer B → Partition 2
Consumer C → Partition 3

If Consumer B crashes:

Rebalance → Consumer A or C picks up Partition 2
Consumer reads from last committed offset (no message loss, some re-processing)

Exactly-Once Semantics (EOS) — Kafka's Complex Feature

Producer idempotency: each producer gets a PID + sequence number
→ Broker detects and deduplicates retried messages

Transactions: producer can atomically write to multiple partitions
→ All writes visible at once to transactional consumers
→ Enables exactly-once in Kafka Streams: read-process-write atomically

Config:

enable.idempotence=true
transactional.id=my-producer-1
isolation.level=read_committed (consumer)

18. Kafka vs RabbitMQ vs SQS

<i>Dimension</i>	<i>Kafka</i>	<i>RabbitMQ</i>	<i>AWS SQS</i>
Message model	Log (ordered, replayable)	Queue (consume-and-delete)	Queue (consume-and-delete)
Retention	Configurable (days to forever)	Until consumed (or TTL)	14 days max
Replay	Yes — any consumer can re-read	No — gone once consumed	No
Throughput	Millions msg/sec	Thousands–low millions	Thousands (per queue)
Ordering	Per partition	Per queue (FIFO queues)	Best-effort (FIFO queues)
Consumer model	Pull (poll)	Push or pull	Pull
Scalability	Horizontal (add partitions)	Horizontal with clustering	Fully managed, auto-scale
Operational complexity	High	Medium	Low (managed)

Choose Kafka when: high throughput, replay needed, event streaming, audit log

Choose RabbitMQ when: complex routing (fanout, headers, dead-letter queues), low latency push delivery

Choose SQS when: simple queue, low operational overhead, AWS ecosystem

Kafka: Advantages vs Disadvantages

<i>Advantages</i>	<i>Disadvantages</i>
Zero-copy I/O (<code>sendfile</code>): consumers read at near-wire speed without CPU overhead	High operational complexity: ZooKeeper (pre-3.0) / KRaft, broker config tuning, partition management
Sequential I/O only: saturates disk bandwidth with no seeks	Replayability is a double-edged sword: consumers that fall behind must process old messages or reset
Replay: any consumer can re-read messages from any offset	Ordering only within partition: global ordering requires 1 partition → limits parallelism
Fan-out: many consumer groups independently read the same topic	Consumer group rebalancing: adding/removing consumers pauses consumption during rebalance
Durable by default: messages written to disk and replicated	Latency floor: batching adds ms of latency; not ideal for sub-millisecond use cases

Advantages	Disadvantages
Scales horizontally: add partitions/brokers without redesigning	No per-message TTL: either all messages in a topic expire together or use log compaction
Log compaction: per-key latest-value retention for state use cases	Exactly-once adds complexity: transactions + idempotent producers require careful configuration
OS page cache acts as transparent in-memory layer	Small message overhead: each message has a fixed metadata overhead; tiny messages are inefficient

19. Exactly-Once Delivery — The Hard Problem

Why It's Hard

```

Producer sends message → Network timeout
Producer doesn't know: did broker receive it or not?
Producer retries → Broker may process it twice

```

To achieve exactly-once, you need:

1. Idempotent producer (deduplicate retries at broker)
2. Idempotent consumer (safe to process same message twice – result is the same)
3. OR transactional processing (atomic consume + process + produce)

Practical Approach: At-Least-Once + Idempotent Consumer

Most systems combine:

- At-least-once delivery** (easier to guarantee, just retry)
- Idempotent processing** (consumer handles duplicates safely)

Idempotency techniques:

```

-- Database insert with idempotency key
INSERT INTO processed_events (event_id, result)
VALUES (?, ?)
ON CONFLICT (event_id) DO NOTHING;
-- If event_id already exists, no-op. Safe to retry.

```

Idempotency Key Patterns

- Use a natural unique key from the message (*orderid*, *transactionid*)
- If no natural key: producer generates UUID and includes it

- Store processed IDs in Redis with TTL (dedup window = max retry window + buffer)

Delivery Guarantee Trade-offs

<i>Guarantee</i>	<i>How Achieved</i>	<i>Advantages</i>	<i>Disadvantages</i>
At-most-once	Send and forget; no retry	Lowest latency; simplest implementation	Message loss possible on failure
At-least-once	Retry until ACK received	No message loss; simple to implement	Duplicates possible; consumer must be idempotent
Exactly-once	Idempotent producer + transactional consumer	No duplicates, no loss	Highest complexity and latency overhead; ~20–30% throughput reduction in Kafka EOS

Practical recommendation: Design for at-least-once delivery with idempotent consumers. True exactly-once is expensive and usually unnecessary if consumers are idempotent. Most financial systems use at-least-once + dedup table rather than true exactly-once.

20. Backpressure

What It Is

A mechanism for a slow consumer to signal to a fast producer to slow down, preventing unbounded queue growth and out-of-memory crashes.

Without Backpressure

```

Producer: 100,000 msg/sec
Consumer: 10,000 msg/sec
Queue: grows at 90,000 msg/sec → fills memory → OOM crash → data loss

```

Backpressure Mechanisms

Reactive Streams / Project Reactor:


```
Flux.range(1, 1_000_000)
  .onBackpressureDrop() // drop if consumer can't keep up
  // OR
  .onBackpressureBuffer(1000) // buffer up to 1000
  // OR
  .onBackpressureLatest() // keep only latest value
```

TCP backpressure:

TCP's receive window is built-in backpressure. If receiver buffer fills, window size $\rightarrow 0$, sender stops.

Kafka consumer backpressure:

```
max.poll.records = 500    // fetch at most 500 records per poll
max.poll.interval.ms = 300000 // if processing takes > 5min, trigger rebalance
fetch.max.bytes = 52428800 // 50MB max per fetch
```

Service-to-service: Return HTTP 429 Too Many Requests with Retry-After header. Caller implements exponential backoff.

Backpressure Strategies: Advantages vs Disadvantages

Strategy	Advantages	Disadvantages	Best For
Drop (onBackpressureDrop)	Never blocks; lowest latency; no memory growth	Data loss — some messages permanently lost	Metrics, sensor readings where latest > completeness
Buffer (onBackpressureBuffer(N))	No data loss up to buffer size	Memory bounded but still grows; OOM if burst exceeds N	Bursty traffic with known peak duration
Latest (onBackpressureLatest)	Always has current state; low memory	Intermediate states lost; only final value kept	UI refresh rates, sensor state (only latest matters)
Block/Slow producer	Zero data loss; back-pressure propagated upstream	Can cause upstream stalls; latency spikes	Financial transactions, ordered processing pipelines
Bounded queue + reject (HTTP 429)	Predictable latency; protects server	Requests dropped; client must retry	API rate limiting, ingestion APIs

21. Zero-Copy I/O

The Four Copies Problem (Detailed)

Traditional read() + write() (e.g., serving a file over a socket):

User space: ■ Kernel space:

 ■

 ■ 1. DMA: Disk → Kernel read buffer (page cache)

app_buffer ← ■■■■■■■■ 2. CPU copy: Kernel read buffer → app_buffer

 ■

app_buffer ■■■■■■■■ → 3. CPU copy: app_buffer → Kernel socket buffer

 ■ 4. DMA: Socket buffer → NIC

 ■

 ■ Context switches: 4 (2 for read, 2 for write)

 ■ CPU copies: 2

sendfile() — Zero User-Space Copy

1. DMA: Disk → Kernel read buffer (page cache)

2. CPU copy: Kernel read buffer → Kernel socket buffer
(in Linux 2.4+: if NIC supports scatter/gather DMA, this copy eliminated too)

3. DMA: Socket buffer → NIC

Context switches: 2 (one syscall)

CPU copies: 1 (or 0 with NIC scatter/gather)

mmap() — Another Approach

mmap() maps file into process virtual address space

No copy needed to "read" – process accesses file data directly from page cache

write() to socket still requires one copy (page cache → socket buffer)

Useful when: app needs to process data (can't use sendfile) but wants zero read copy

Applications Beyond Kafka

- Nginx^{**}: Uses sendfile() for static file serving
- Java NIO^{**}: FileChannel.transferTo() maps to sendfile()
- GPU computing^{**}: RDMA (Remote Direct Memory Access) extends zero-copy to network-attached memory

Zero-Copy: Advantages vs Disadvantages

Advantages	Disadvantages
Eliminates 2 CPU copies and 2–4 context switches per I/O operation	Cannot transform data in transit: sendfile() sends bytes as-is; any encryption/compression requires a copy

Advantages	Disadvantages
Saturates NIC bandwidth without CPU bottleneck (multi-GB/s on a single core)	Platform-specific: <code>sendfile()</code> is Linux; <code>sendfile()</code> semantics differ on macOS; Windows uses <code>TransmitFile()</code>
Reduces CPU utilization for I/O-bound workloads: frees CPU for business logic	NIC scatter/gather DMA support required for the truly zero-copy path (most modern NICs support this)
Reduces memory bus pressure: no data copied through CPU cache	Only benefits file-to-network path: does not help with network-to-memory or in-memory transformations
Critical for high-throughput systems: Kafka can saturate 10GbE NIC on commodity hardware	Debugging is harder: cannot inspect data in transit as easily as with user-space buffer
Works transparently with OS page cache: hot files served at memory speed	Application must manage file descriptors carefully; incorrect usage can cause data corruption

22. HTTP/2 vs HTTP/3 (QUIC)

HTTP/1.1 Problems

- Head-of-line blocking**: requests processed serially per connection
- Workaround**: browsers open 6 connections per domain (wasteful)
- Text-based headers: verbose, no compression

HTTP/2 Improvements

Multiplexing: multiple requests/responses over single TCP connection
→ Stream 1, Stream 2, Stream 3 all in-flight simultaneously
→ No head-of-line blocking at application layer

Header compression (HPACK): headers compressed using shared dictionary
→ 85-95% reduction in header bytes on repeated requests

Server push: server can push resources client will need before client asks
→ Not widely used in practice

Binary framing: more efficient than text

Remaining problem: TCP head-of-line blocking

If one TCP packet is lost, all HTTP/2 streams stall until that packet is retransmitted. Loss rate of 2% can negate HTTP/2 multiplexing benefits.

HTTP/3 — QUIC (UDP-based)

QUIC runs over UDP, not TCP.
Implements its own reliability, flow control, and congestion control.

Key benefits:

1. Stream independence: packet loss in Stream 1 doesn't block Stream 2
2. 0-RTT connection establishment: client resumes connection with 0 round trips (TLS 1.3 session resumption built-in)
3. Connection migration: connection identified by Connection ID, not IP:port
 - Switch from Wi-Fi to cellular without reconnecting
 - APNs uses QUIC for this exact reason

Apple adoption: Safari, APNs, iCloud, Maps all use HTTP/3/QUIC

HTTP/1.1 vs HTTP/2 vs HTTP/3: Advantages vs Disadvantages

<i>Dimension</i>	<i>HTTP/1.1</i>	<i>HTTP/2</i>	<i>HTTP/3 (QUIC)</i>
Head-of-line blocking	Per-request (each request blocks next)	Application-layer solved; TCP-layer HOL remains	Fully eliminated per stream
Connection reuse	Multiple connections (6 per domain)	Single connection, multiplexed	Single connection, multiplexed
Header overhead	Large (uncompressed text)	Compressed (HPACK)	Compressed (QPACK)
Connection setup	1 RTT TCP + 1–2 RTT TLS = 2–3 RTT	Same as HTTP/1.1	0-RTT on reconnect (session resumption)
Connection migration	No (tied to IP:port)	No	Yes (Connection ID based)
Packet loss impact	One request blocked	All streams stalled	Only affected stream stalled
Load balancer complexity	Simple	Medium	Complex (connection ID routing)
Middlebox compatibility	Universal	Good	Some firewalls block UDP/QUIC
Debugging	Easy (text, wireshark)	Binary, needs tooling	Binary + UDP, harder to inspect
Browser support	Universal	Universal	95%+ modern browsers

When HTTP/2 is enough: most API servers behind a low-loss LAN-adjacent load balancer — TCP HOL rarely triggers.

When HTTP/3 matters: mobile clients on lossy networks, real-time low-latency (APNs, gaming, video), connection migration use cases.

23. gRPC Internals

What It Is

RPC framework by Google. HTTP/2 + Protocol Buffers (protobuf).

Why It's Faster Than REST+JSON

Protobuf vs JSON:

```
JSON: {"user_id": 12345, "name": "Alice", "active": true}
→ Text encoding, field names repeated every message
→ ~50 bytes

Protobuf: binary encoding, field numbers instead of names
→ field 1 (user_id): varint 12345 = 3 bytes
→ field 2 (name): length-prefixed "Alice" = 7 bytes
→ field 3 (active): varint 1 = 2 bytes
→ ~12 bytes (4x smaller)
→ No parsing overhead (schema-driven deserialization)
```

HTTP/2 streaming:

```
gRPC streaming types:
- Unary: single request, single response (like HTTP REST)
- Server streaming: one request, stream of responses (e.g., logs subscription)
- Client streaming: stream of requests, one response (e.g., file upload with progress)
- Bidirectional streaming: full duplex (e.g., real-time collaboration)
```

gRPC vs REST

<i>Dimension</i>	<i>gRPC</i>	<i>REST+JSON</i>
Serialization	Binary (protobuf) — fast, compact	Text (JSON) — slow, large
Browser support	Requires grpc-web proxy	Native
Discoverability	Schema required (proto file)	OpenAPI/Swagger
Streaming	Built-in (4 patterns)	Limited (SSE for server push)
Type safety	Enforced by proto schema	Optional (OpenAPI)

<i>Dimension</i>	<i>gRPC</i>	<i>REST+JSON</i>
Use case	Service-to-service	Public APIs, browser clients

gRPC: Advantages vs Disadvantages

<i>Advantages</i>	<i>Disadvantages</i>
4x smaller payload than JSON: reduces bandwidth and serialization CPU cost	Not human-readable: binary protobuf requires tooling to inspect (grpcurl, protoc)
Strongly typed contract: .proto schema shared between client and server	Browser requires grpc-web proxy (Envoy) — adds infrastructure complexity
Code generation: client stubs auto-generated in 10+ languages from .proto	Schema required: both sides must have .proto; REST is more flexible for ad-hoc clients
Built-in streaming: 4 streaming patterns without custom protocol work	Protobuf schema evolution requires discipline: field numbers must never be reused
HTTP/2 multiplexing: multiple concurrent RPCs on one connection	Harder to debug than REST: can't curl a gRPC endpoint directly
Load balancer compatibility: L7-aware load balancers needed (not L4 TCP)	Learning curve for teams used to REST/HTTP
Deadline propagation: client sets timeout, propagated across service hops automatically	Server push / server streaming less familiar to ops teams

24. TCP vs UDP — When Engineers Get This Wrong

The Wrong Mental Model

"UDP is unreliable so you only use it for video games and DNS."

The Right Mental Model

UDP gives you a **blank canvas**. You implement exactly the guarantees you need:

TCP gives you:

- Ordered delivery
- Reliable delivery (retransmission)
- Congestion control
- Flow control
- Connection state
- Head-of-line blocking (consequence of ordering)

UDP gives you:

- Minimal header (8 bytes vs 20+ for TCP)
- No connection state
- No head-of-line blocking
- You implement what you need on top

Real Applications of UDP

QUIC (HTTP/3): Implements reliability + ordering per stream over UDP. Gets rid of TCP's cross-stream head-of-line blocking.

WebRTC: Real-time media. A dropped audio packet is better to skip than to delay the stream for retransmission.

DNS: Query-response fits in one datagram. No need for connection overhead.

Game networking: Custom reliability (only retransmit critical state, drop stale position updates).

Financial trading: Some HFT systems use raw UDP (multicast) for market data because TCP's congestion control introduces jitter.

TCP vs UDP: Advantages vs Disadvantages

<i>Dimension</i>	<i>TCP</i>	<i>UDP</i>
Reliability	Built-in: retransmits lost packets	None built-in: lost packets are lost
Ordering	Guaranteed in-order delivery	No ordering guarantee
Head-of-line blocking	Yes: all data stalls behind a lost packet	No: each datagram independent
Connection overhead	3-way handshake: ~1 RTT setup	No handshake: first packet is first data
Header size	20+ bytes	8 bytes
Congestion control	Built-in (CUBIC, BBR): adapts to network	None: you implement or accept congestion

<i>Dimension</i>	<i>TCP</i>	<i>UDP</i>
Broadcast/multicast	Not supported	Supported: one packet to many receivers
Latency floor	Higher: ACK round-trips, buffers	Lower: no buffering, no ACK wait
Suitable for	File transfer, HTTP, databases, SSH	Real-time media, DNS, game state, QUIC
Implementation complexity	Low (OS handles everything)	Higher (must implement needed guarantees)

Key insight: UDP is not "unreliable TCP." It's a blank canvas. QUIC, WebRTC, and game protocols prove you can build reliable, ordered, congestion-controlled protocols on UDP that outperform TCP for specific workloads.

25. Connection Pooling

Why Connections Are Expensive

Opening a database connection:

1. TCP 3-way handshake (~0.5 RTT)
2. TLS negotiation (~1-2 RTTs)
3. DB authentication (1 RTT)
4. Session setup (memory allocation on DB server)

Total: ~5-10ms per connection

DB server: 100 connection limit (memory, file descriptors)

Without pooling: $1000 \text{ req/sec} \times 10\text{ms/conn} = 10,000$ concurrent connections → DB crashes

Connection Pool Design

Pool size: typically $2 \times$ (CPU cores on DB server) for compute-bound workloads

Or: Little's Law: $\text{pool_size} = \text{avg_latency_seconds} \times \text{target_throughput_per_db}$

Pool behavior:

- Request comes in → borrow connection from pool (if available)
- If pool empty → wait in queue (configurable timeout)
- Request completes → return connection to pool

Connection validation:

- On borrow: test-on-borrow ("SELECT 1") – safe but adds latency
- Keepalive: periodic heartbeat to detect dead connections

PgBouncer (PostgreSQL Connection Pooler)

Transaction mode pooling: connection returned to pool after each transaction
→ 10,000 app connections map to ~100 DB connections
→ Most efficient, incompatible with session-level features (prepared statements, SET vars)

Session mode: connection held for entire client session
→ Less multiplexing, safer

PgBouncer sits between app servers and PostgreSQL:
App → PgBouncer → PostgreSQL

Advantages vs Disadvantages

<i>Advantages</i>	<i>Disadvantages</i>
Eliminates per-request connection overhead (5–10ms saved per request at cold-connection rate)	Pool exhaustion: if all connections in use, new requests wait or timeout — must size pool correctly
Caps total DB connections: 10,000 app threads → 100 DB connections (DB server no longer overwhelmed)	Connection leaks: if code doesn't return connection to pool on exception, pool drains silently
Reuse warm connections: no TCP/TLS/auth handshake on each request	Session-level features incompatible with transaction-mode pooling (prepared statements, advisory locks)
Reduces DB server memory: each idle connection consumes ~5MB on PostgreSQL	Pool is a shared resource: one slow query holds a connection, reducing availability for others
Little's Law optimal sizing: $\text{pool_size} = \text{latency} \times \text{throughput}$	Wrong pool size: too small → queueing; too large → DB CPU overwhelmed by context switching
Easily tunable: max pool size, wait timeout, idle timeout configurable	In-process pools don't help if bottleneck is on DB server side (need PgBouncer or similar proxy)

26. CAP Theorem — The Nuanced View

What It Actually Says

During a **network partition** (nodes can't communicate), you must choose:

- Consistency^{**}: every read receives the most recent write (or an error)
- Availability^{**}: every request receives a response (may be stale)

CAP does NOT say you always sacrifice one of three. **In the absence of partitions, you can have both C and A.** The choice only matters when a partition occurs.

Real Systems

CP systems (prefer consistency over availability during partition):
HBase, Zookeeper, etcd, Spanner
→ During partition: some nodes reject requests rather than serve stale data

AP systems (prefer availability during partition):
Cassandra, CouchDB, DynamoDB (default config)
→ During partition: serve potentially stale data, reconcile later

CA systems (only possible without partitions – single-node or LAN):
Single-node PostgreSQL, MySQL (not distributed)

27. PACELC — The Better Model

Why CAP Is Incomplete

CAP only addresses behavior during partitions. But **partitions are rare** — what about normal operation?

PACELC: **P**artition → **A**vailability vs **C**onsistency; **E**lse → **L**atency vs **C**onsistency

During partition (P):
Choose: Availability (A) or Consistency (C)

Else (normal operation, E):
Choose: Latency (L) or Consistency (C)
(Strong consistency requires coordination = higher latency)

PACELC Classification

System	P choice	E choice
DynamoDB (eventual)	PA	EL
Cassandra	PA	EL
Spanner	PC	EC
CRDT-based systems	PA	EL
PostgreSQL (single-node)	N/A	EC

Key insight: Spanner achieves PC+EC (strong consistency with low latency) by using atomic clocks (TrueTime API) to reduce coordination latency. This is what GPS-synchronized atomic clocks buy at Google scale.

28. Vector Clocks & Causal Consistency

Logical Clocks (Lamport Clocks)

```
Each node has a counter C.  
On local event: C++  
On send: attach C, C++  
On receive: C = max(C_local, C_received) + 1
```

This establishes "happened-before" ordering but not causality between concurrent events.

Vector Clocks

```
N nodes, each node tracks a vector: [0, 0, 0] (one entry per node)
```

Node A sends message: A's vector = [3, 0, 0] → sent to B
Node B receives: B's vector = [0, 1, 0] → merge = [3, 1, 0], increment = [3, 2, 0]

Comparison:
V1 < V2 if: every entry of V1 ≤ V2, and at least one is strictly less
Concurrent: neither V1 < V2 nor V2 < V1 → conflict!

Amazon Dynamo uses vector clocks to detect concurrent writes → surfaces conflicts to application layer.

Vector Clocks: Advantages vs Disadvantages

Advantages	Disadvantages
Detects concurrent writes exactly — no false conflict reports	Vector size grows with number of nodes: $O(N)$ space per event in an N-node cluster
Identifies causal ordering: $V1 < V2$ means V1 happened-before V2 with certainty	Garbage collection required: stale node entries must be pruned or vector clocks grow unboundedly
No coordination required: nodes update clocks locally and exchange on sync	Conflict resolution is left to application: detecting conflict ≠ resolving it
Enables eventual consistency with conflict detection (used in DynamoDB, Riak)	Pruning causes false conflicts: if a node's entry is pruned and it later writes, it looks concurrent

<i>Advantages</i>	<i>Disadvantages</i>
Captures full causality: can reconstruct which write caused which	Version vector overhead: every stored value must carry a vector clock

29. CRDTs

Convergent Replicated Data Types

Data structures designed so that **any number of concurrent updates will converge to the same final state** when merged, without coordination.

G-Counter (Grow-only counter):
 Each node increments only its own slot in a vector
 Merge = take max of each slot
 Value = sum of all slots

Node A: [3, 0, 0]

Node B: [0, 5, 0]

Merged: [3, 5, 0] → value = 8

Same result regardless of merge order ✓

Types of CRDTs

<i>Type</i>	<i>Operations</i>	<i>Example Use</i>
G-Counter	increment only	View count
PN-Counter	increment/decrement	Cart item count
G-Set	add only	Tag set
2P-Set	add/remove (once)	Membership (no re-add after remove)
OR-Set	add/remove (re-add ok)	Shopping cart items
LWW-Register	assign (last-write-wins)	User profile field
MV-Register	concurrent updates stored	Collaborative text cursor
RGA / TreeDoc	insert/delete in sequence	Collaborative text editing

Apple Notes, Google Docs, Figma all use CRDT-based approaches for collaborative editing.

CRDTs: Advantages vs Disadvantages

<i>Advantages</i>	<i>Disadvantages</i>
No coordination required: merge is always correct without locks or consensus	Limited data model: not all data structures have a CRDT variant — complex relational structures are hard
Always available: any node can accept writes even during partition	Merge semantics can be surprising: e.g., OR-Set allows re-adding deleted items, which may violate app logic
Convergent by design: any merge order produces the same result	Tombstone accumulation: deleted elements stored as tombstones forever (or require expensive GC)
Natural fit for distributed, offline-first systems (sync when reconnected)	High metadata overhead for some types: G-Counter requires $O(N)$ metadata per counter
Eliminates conflict resolution code in the application	Not suitable for operations with strong invariants (e.g., "quantity must never go negative")
Enables multi-master writes: any replica accepts writes without coordination	Implementation complexity: correct CRDT implementations are subtle; off-the-shelf libraries needed

30. Optimistic vs Pessimistic Locking

Pessimistic Locking

```
-- Lock row before reading; hold until transaction completes
BEGIN;
SELECT * FROM inventory WHERE product_id = 1 FOR UPDATE; -- acquires row lock
UPDATE inventory SET quantity = quantity - 1 WHERE product_id = 1;
COMMIT; -- releases lock
```

When to use: High contention (many concurrent writers for same row), operations where conflicts are likely. Example: airline seat reservation.

Optimistic Locking

```
-- Read with version number; update only if version hasn't changed
SELECT quantity, version FROM inventory WHERE product_id = 1;
-- version = 5

UPDATE inventory
SET quantity = quantity - 1, version = version + 1
WHERE product_id = 1 AND version = 5; -- fails if someone else updated first

-- If 0 rows affected → conflict → retry
```

When to use: Low contention (conflicts are rare), read-heavy workloads, distributed systems where locks are expensive.

Compare-and-Swap (CAS) — Hardware-Level Optimistic Lock

```
atomic { if (memory[addr] == expected) { memory[addr] = new_value; return success; } else return fail; }
```

Used in lock-free data structures. Redis WATCH + MULTI/EXEC implements application-level CAS.

Optimistic vs Pessimistic: Advantages vs Disadvantages

<i>Dimension</i>	<i>Pessimistic Locking</i>	<i>Optimistic Locking</i>
Best for	High-contention writes (many conflicts expected)	Low-contention reads with occasional writes
Throughput	Lower — lock acquisition serializes concurrent writers	Higher — no waiting; readers never block
Latency	Higher — lock wait time adds to request latency	Lower in no-conflict case; higher when retry needed
Deadlock risk	Yes — multiple locks in different order → deadlock	No — no locks held
Implementation	Simpler — DB handles locking automatically	More complex — requires version column + retry logic
Starvation	Possible — a writer can starve if reads hold locks	Possible — a writer retries indefinitely in high-contention
Distributed systems	Very expensive — distributed locks require coordination	Natural fit — version check in the DB is local
Data integrity	Guaranteed — no concurrent modification possible	Only guaranteed at commit — window of possible conflict

Rule of thumb: If conflict probability > ~30%, pessimistic is usually faster due to fewer retries. If < ~10%, optimistic wins on throughput.

31. CPU Cache Hierarchy & Cache-Friendly Code

Cache Levels (Approximate, Modern CPU)

```
L1 Cache: 32-64 KB, 1-4 ns latency (per core)
L2 Cache: 256 KB - 1 MB, 5-12 ns latency (per core)
L3 Cache: 8-64 MB, 30-50 ns latency (shared across cores)
DRAM: 4-100 GB, 100-200 ns latency
NVMe SSD: 100-200 µs latency (1000x slower than DRAM)
HDD: 5-10 ms latency (50,000x slower than DRAM)
```

Cache Line

CPU fetches data in **64-byte cache lines**, not individual bytes.

Cache-unfriendly: column-major access of row-major array

```
// Java 2D array stored row-major
int[][] matrix = new int[1000][1000];

// Cache-unfriendly: stride of 1000 × 4 bytes = 4000 bytes
// Each access is a cache miss
for (int j = 0; j < 1000; j++)
    for (int i = 0; i < 1000; i++)
        sum += matrix[i][j];

// Cache-friendly: sequential access within each row
for (int i = 0; i < 1000; i++)
    for (int j = 0; j < 1000; j++)
        sum += matrix[i][j];
```

Cache-friendly version: ~5–10x faster on modern CPUs.

False Sharing (Multi-Core)

Two threads write to different variables that share a cache line:

```
struct {
    long counter_a; // bytes 0-7
    long counter_b; // bytes 8-15
} // both fit in one 64-byte cache line
```

Thread 1 writes counter_a → invalidates cache line → Thread 2's counter_b also evicted

Thread 2 must re-fetch from L3/DRAM even though it never touched counter_a!

Fix: pad to separate cache lines

```
struct {
    long counter_a;
    long padding[7]; // pad to 64 bytes
    long counter_b;
}
```

Java's `@Contended` annotation handles this automatically (restricted to JDK internals by default).

32. Memory-Mapped Files (mmap)

What It Is

Map a file into the process's virtual address space. Accessing mapped memory = accessing file data directly from page cache. The OS handles I/O transparently.

```
void *addr = mmap(NULL, file_size, PROT_READ, MAP_SHARED, fd, 0);
// Now access file like a byte array:
char first_byte = ((char*)addr)[0]; // if not in page cache: triggers page fault → OS loads page
```

Benefits

- No explicit `read()` calls**: access file data with pointer arithmetic
- Zero copy for reads**: file bytes are in page cache; process reads directly (no copy to user buffer)
- Shared between processes**: multiple processes can `mmap` the same file, sharing the same page cache pages
- Lazy loading**: only pages that are accessed are loaded into RAM (demand paging)

Used By

- SQLite: `mmaps` the database file for read access
- RocksDB: `mmaps` SSTable index blocks
- Elasticsearch: `mmaps` Lucene index segments
- JVM: `java.nio.MappedByteBuffer`

Risks

- Not suitable for writes on large files**: a write to a mapped region triggers a copy-on-write page fault, can't control which pages are dirty
- Process crash = no explicit close/flush**: mapped writes may not be durable unless `msync()` is called
- Virtual address space exhaustion**: on 32-bit systems, can't mmap large files (64-bit resolves this)

mmap: Advantages vs Disadvantages

<i>Advantages</i>	<i>Disadvantages</i>
Zero-copy reads: file data accessed directly from page cache, no user-space copy	Page fault latency: first access to unmapped page causes page fault (~1μs) — unpredictable latency
Lazy loading: only touched pages are loaded into RAM — efficient for sparse access	OOM killer risk: large mmap regions count against virtual memory; kernel may kill process under memory pressure
Shared between processes: same physical pages shared across multiple processes (no duplication)	Durability risk: writes to mmap require explicit <code>msync()</code> — easy to forget, leading to data loss on crash
Simple programming model: file access via pointer, no read/write syscalls	Cannot use with files being written by another process without coordination
OS handles prefetching and eviction automatically	TLB pressure: many small mmap regions fill the TLB, degrading performance (prefer fewer large regions)
Works well for read-heavy, random-access patterns (e.g., database index lookups)	Not suitable for append-heavy workloads: extending a file requires re-mmap or <code>mremap</code>

33. Copy-on-Write (COW)

OS-Level COW (fork())

`fork()` creates a child process. Naively, all parent memory would be copied.

COW optimization:

- Child shares parent's memory pages (marked read-only)
- On write by either process: OS copies just that page (4KB) before allowing write
- Only pages that diverge are actually copied

Result: `fork()` is $O(1)$ regardless of process size

Benefit: Redis uses `fork()` for snapshots — parent serves traffic while child writes RDB file

Language-Level COW (Swift, Rust)

```
// Swift value types use COW
var a = [1, 2, 3, 4, 5] // Array allocated, reference count = 1
var b = a              // b points to same storage, reference count = 2 — NO COPY yet
b.append(6)            // reference count > 1, so copy-on-write: b gets its own copy now

// Before append: O(1) copy (just increment reference count)
// After mutation: O(n) copy (copy the storage)
```

Swift's Array, Dictionary, String, and Set all use COW. This makes value semantics efficient — you pay for copies only when you actually mutate.

COW: Advantages vs Disadvantages

<i>Advantages</i>	<i>Disadvantages</i>
fork() is O(1): large process forks instantly regardless of memory size	Copy-on-write copies still happen: if child writes many pages, memory usage doubles temporarily
Memory deduplication: parent and child share identical pages, reducing RAM usage	Latency spikes: the first write to a shared page triggers a page copy (~1μs each) — unpredictable
Enables Redis RDB snapshots with zero downtime: fork + child writes, parent keeps serving	"Fork bomb" risk: if child writes aggressively, parent's available memory shrinks rapidly
Language-level COW (Swift): value semantics with reference performance for non-mutating operations	Swift COW: only works for standard library types; custom types require manual <code>isKnownUniquelyReferenced()</code> check
Prevents accidental shared mutation: in Swift, mutating a copy doesn't affect the original	Unexpected mutation cost: a "simple" append on a shared Swift Array can trigger O(n) copy
Safe multi-process data sharing: read-only sharing is free; writes are safely isolated	Complexity: programmers must understand when COW triggers to avoid surprising performance cliffs

34. Epoll / kqueue — I/O Multiplexing

The C10K Problem

How do you handle 10,000 simultaneous connections?

Naive: one thread per connection

10,000 connections × 1MB stack = 10 GB RAM just for stacks
Context switching overhead: scheduler must switch among 10,000 threads

select() / poll() — O(n) per call

```
fd_set = {fd1, fd2, ..., fd10000}
select(fd_set) → kernel scans ALL file descriptors → returns ready ones
O(n) per call where n = number of watched descriptors
Not scalable beyond ~1000 fds
```

epoll (Linux) / kqueue (BSD/macOS) — O(1) per event

```
epoll_create() → create epoll instance
epoll_ctl(EPOLL_CTL_ADD, fd, event) → register fd (once, O(1))
epoll_wait() → blocks, returns ONLY ready fds (O(k) where k = number ready)

Kernel uses callback-based approach: when fd becomes ready, it's added to ready list
epoll_wait returns immediately with only the ready fds
```

This enables **event-driven, single-threaded servers** (Node.js, Nginx, Redis) to handle millions of connections with minimal resources.

Nginx vs Apache Model

Apache: one thread/process per connection → limited concurrency
Nginx: single-threaded event loop + epoll → handles 10K+ connections per worker
→ 1 Nginx worker handles more connections than 1000 Apache workers

epoll vs Alternatives: Advantages vs Disadvantages

<i>Dimension</i>	<i>Thread-per-Connection</i>	<i>select/poll</i>	<i>epoll/kqueue</i>
Scalability	Poor (1 thread = ~1MB stack)	Poor O(n) per call	Excellent O(1) per event
Max connections	~1000–10,000	~1000 (fd_set limit)	Millions
CPU usage	High (context switching)	High (scan all fds)	Low (only ready fds)
Simplicity	Simple (blocking I/O)	Medium	Complex (event loop required)

<i>Dimension</i>	<i>Thread-per-Connection</i>	<i>select/poll</i>	<i>epoll/kqueue</i>
Blocking operations	Transparent (whole thread blocks)	Must be non-blocking	Must be non-blocking
CPU-bound tasks	Good (parallel threads)	N/A	Bad (blocks event loop)
Library/framework required	None	None	Yes (libuv, Netty, Tokio, etc.)
Debugging	Easy (stack trace per request)	Medium	Hard (callback chains, async stack)

Key limitation of epoll: CPU-bound work in the event loop blocks all other connections. Must offload to thread pool (Node.js worker_threads, Java Netty with separate business logic threads).

35. JVM GC — What a Senior Engineer Must Know

GC Generations

```

Heap regions:
  Young Gen (Eden + S0 + S1): short-lived objects; collected frequently (minor GC)
  Old Gen (Tenured): long-lived objects; collected infrequently (major/full GC)

Object promotion:
  Eden → survives minor GC → Survivor (S0/S1) → survives N minor GCs → Old Gen
  Threshold controlled by: -XX:MaxTenuringThreshold (default 15)

```

G1GC (Default in Java 9+)

```

Heap divided into equal-sized regions (~2MB)
Any region can be Eden, Survivor, Old, or Humongous (large objects)

G1GC goals:
- Predictable pause times (default target: 200ms)
- Concurrent marking (while app runs)
- Incremental collection (collect regions with most garbage first)

Key parameters:
-XX:MaxGCPauseMillis=200    // pause time goal
-XX:G1HeapRegionSize=2m     // region size
-Xmx8g -Xms8g               // set min=max to avoid heap resizing pauses

```

ZGC (Java 15+) / Shenandoah — Sub-Millisecond Pauses

ZGC:

- All GC phases concurrent (app continues running)
- Pauses < 1ms (even for 100GB+ heaps)
- Uses colored pointers + load barriers to track references during concurrent compaction

Trade-off:

- Higher CPU overhead (concurrent work steals CPU from app threads)
- Higher memory overhead
- Best for: latency-sensitive apps with large heaps (real-time systems, trading)

GC Tuning Indicators

GC log analysis:

- Frequency of full GC → if frequent: heap too small or memory leak
- Pause time > SLO → switch to ZGC or tune G1 pause target
- Humongous allocations → large objects bypass young gen, cause full GCs
- Promotion failure → old gen full, increase -Xmx or reduce object retention

Tools: GCViewer, GCEasy, JVM profiler (JFR / async-profiler)

GC Algorithm Comparison: Advantages vs Disadvantages

Algorithm	Advantages	Disadvantages	Best For
Serial GC	Simplest; lowest overhead for small heaps	Stop-the-world pauses; single-threaded collection	Small apps, batch jobs, low-memory environments
Parallel GC	High throughput; multiple GC threads	Long stop-the-world pauses; poor latency	Batch processing, throughput-optimized workloads
G1GC (default Java 9+)	Predictable pause times (target: 200ms); incremental collection	Complex tuning; can miss pause targets under pressure	General-purpose; API servers; most production JVM apps
ZGC (Java 15+)	Sub-millisecond pauses even on 100GB+ heaps; concurrent compaction	~10–15% CPU overhead; higher memory overhead	Latency-sensitive: trading systems, real-time APIs
Shenandoah (OpenJDK)	Sub-millisecond pauses; concurrent evacuation	Similar CPU overhead to ZGC; less mature	Low-latency applications on OpenJDK

Key insight: GC is a trade-off triangle — **throughput vs. latency vs. memory overhead**. G1GC is the pragmatic default. ZGC is the right choice when your P99 SLO is < 10ms and you can afford the CPU cost.

This is a living reference. Each pattern here is a vocabulary item for Staff-level system design conversations. Knowing not just "what" but "why" and "what are the trade-offs" is what separates a staff answer from a senior answer.